Time module in python

The `time` module in Python provides various time-related functions. It allows you to work with time in terms of **seconds**, **delays**, and **date/time formatting**.

Commonly Used Functions in `time` Module:

Function	Description
`time.time()`	Returns the current time in seconds since the epoch (Unix time)
`time.sleep(seconds)`	Suspends execution for the given number of seconds
`time.localtime([secs])`	Converts seconds since epoch to a time tuple in local time
`time.gmtime([secs])`	Converts seconds since epoch to UTC time
`time.strftime(format,t)`	Converts a time tuple to a string as per format

Example Usage:

```
import time

# Current time in seconds
print("Epoch time:", time.time())

# Delay for 2 seconds
print("Wait for 2 seconds...")
time.sleep(2)
print("Resumed!")

# Local time
local = time.localtime()
print("Local time tuple:", local)
print("Formatted:", time.strftime("%Y-%m-%d %H:%M:%S", local))

# Convert string to time
time_str = "2025-07-16 11:00:00"
parsed = time.strptime(time_str, "%Y-%m-%d %H:%M:%S")
print("Parsed time tuple:", parsed)

# Readable time from timestamp
print("CTime:", time.ctime())
```

Notes:

- **Epoch**: Time counted from 00:00:00 UTC on 1 January 1970.
- `sleep()` is useful for **delays**, animations, and **rate limiting**.
- `strftime()` and `strptime()` use formatting codes like:
  - `%Y` = Year, `%m` = Month, `%d` = Day
  - `%H` = Hour, `%M` = Minute, `%S` = Second

Command-Line Utility in python

In Python, a **Command-Line Utility** is a script or program that runs from the terminal/command prompt and accepts **arguments** and **options** like a regular system command.

Benefits:

- Automates tasks (e.g., file renaming, data processing)
- Can be packaged and installed like Linux commands.
- Works well for scripting and devops.

Simple CLI Utility Using `argparse`

We'll make a utility that adds or subtracts two numbers via command line.

Step 1: Code (calc.py)

```
import argparse

# Create parser
parser = argparse.ArgumentParser(description="Simple calculator
utility")

# Add arguments
parser.add_argument("num1", type=float, help="First number")
parser.add_argument("num2", type=float, help="Second number")
parser.add_argument("--operation", "-o", choices=["add", "sub"],
default="add", help="Operation to perform")

# Parse arguments
args = parser.parse_args()

# Perform operation
if args.operation == "add":
    result = args.num1 + args.num2
else:
    result = args.num1 - args.num2

print("Result:", result)
```

Step 2: Run in Command Line

```
python calc.py 10 5 --operation add
# Output: Result: 15.0

python calc.py 10 5 -o sub
# Output: Result: 5.0
```

Notes:

- argparse.ArgumentParser() sets up help, usage, etc.
- --operation is an **optional argument** with a default.
- You can add more features like multiplication, logging, file inputs, etc.

Walrus operator in python

The **Walrus Operator (`:=`)** in Python is used for **assignment expressions**. It was introduced in **Python 3.8**.

It lets you **assign a value to a variable as part of an expression** — like inside an `if`, `while`, list comprehension, etc.

Example 1: Traditional vs Walrus

Without Walrus
```
value = input("Enter something: ")
if value != "":
    print("You entered:", value)
```

With Walrus
```
if (value := input("Enter something: ")) != "":
    print("You entered:", value)
```

Example 2: `while` loop

Old Way
```
line = input("Enter text (blank to stop): ")
while line != "":
    print("Line:", line)
    line = input("Enter text (blank to stop): ")
```

With Walrus
```
while (line := input("Enter text (blank to stop): ")) != "":
    print("Line:", line)
```

Caution

- Works only in Python **3.8+**
- Avoid **overusing** it where it reduces code readability
- Don't use it for **multi-variable assignments**

When to Use:

- Inside `if` / `while` conditions to avoid repetition
- In comprehensions where filtering and assigning are both needed
- When you want to keep code **clean and DRY**

Shutil module in python

The `shutil` module in Python provides **high-level file operations**, such as copying, moving, archiving, and removing files or directories.

It's part of the **standard library**, so you don't need to install anything.

Common Functions in `shutil` Module

Function	Description
`shutil.copy(src, dst)`	Copies a file (only contents, not metadata)
`shutil.copy2(src, dst)`	Like `copy` but also copies metadata
`shutil.copytree(src, dst)`	Recursively copies an entire directory tree
`shutil.move(src, dst)`	Moves a file or directory
`shutil.rmtree(path)`	Recursively deletes a directory tree

Example Code

```
import shutil
import os

# Copy a file
shutil.copy("source.txt", "backup.txt")

# Copy file with metadata
shutil.copy2("source.txt", "backup_with_meta.txt")

# Move a file
shutil.move("backup.txt", "folder1/backup.txt")

# Delete a directory
shutil.rmtree("old_folder")

# Create a ZIP archive of a folder
shutil.make_archive("my_backup", "zip", "my_folder")

# Extract archive
shutil.unpack_archive("my_backup.zip", "restored_folder")

# Get disk usage
usage = shutil.disk_usage("/")
print(f"Total: {usage.total}, Used: {usage.used}, Free:
{usage.free}")
Tips
```

- `copy()` works only on **files**, not directories.
- Be careful with `rmtree()` — it deletes everything without confirmation!
- `make_archive()` can save in formats: `"zip"`, `"tar"`, `etc`.

Request module in python

The `requests` module in Python is a popular library used to **send HTTP requests**. It is not included in the standard library, so you usually need to install it using pip:

```
pip install requests
```

What Can `requests` Do?

It allows you to:

- Make HTTP **GET**, **POST**, **PUT**, **DELETE**, and other requests
- Send data in query strings, form data, JSON, and files
- Handle response status, headers, and cookies
- Set headers, authentication, and more

Basic Example: GET Request

```
import requests

response = requests.get('https://api.github.com')
print(response.status_code)       # 200
print(response.text)              # Full response text
(HTML/JSON/etc.)
```

Fetching JSON Data

```
import requests

response =
requests.get("https://jsonplaceholder.typicode.com/users")

data = response.json()

print(data[1]["address"]["street"])
```

POST Request with Data

```
import requests

payload = {'username': 'neeraj', 'password': '1234'}
response = requests.post('https://httpbin.org/post', data=payload)

print(response.status_code)
print(response.json())  # Converts JSON response to Python dict
```

Passing Parameters in URL

```
params = {'q': 'python'}
response = requests.get('https://www.google.com/search',
params=params)

print(response.url)  # Final URL with parameters
```

Custom Headers

```
headers = {'User-Agent': 'my-app'}
response = requests.get('https://httpbin.org/headers',
headers=headers)
print(response.json())
```

Response Properties

Property	Description
`response.text`	Response body as string
`response.json()`	Convert JSON response to Python
`response.status_code`	HTTP status code (e.g., 200)
`response.headers`	HTTP headers as a dictionary

What is a Generator?

A **generator** is a function that uses the `yield` keyword instead of `return`. It "yields" values one at a time and **pauses** after each `yield`, resuming from where it left off.

Example of a Generator Function

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

gen = count_up_to(5)
for number in gen:
    print(number)
```

Output:

```
1
2
3
4
5
```

How is it Different from return?

return	**yield**
Ends the function completely	Pauses and saves the state
Returns a single value	Returns a generator object
Cannot be used for iteration	Can be iterated (like using `for`)

Generator vs List Example

```
# List version (stores all numbers)
def squares_list(n):
    return [i*i for i in range(n)]

# Generator version (generates one at a time)
def squares_gen(n):
    for i in range(n):
        yield i*i
print(squares_list(5))      # [0, 1, 4, 9, 16]
```

```
print(list(squares_gen(5))) # [0, 1, 4, 9, 16]
```

Benefits of Generators

- **Memory-efficient**: Doesn't load everything in memory
- **Lazy evaluation**: Values generated on the fly
- **Useful for streaming data or infinite sequences**

Important Notes

- You can iterate only once over a generator
- Use next(generator) to manually get the next value
- Raise StopIteration automatically when done